

Code review — testare

Complement la CODE_REVIEW.pdf: ce acoperă testele tale și ce ratează.

Branch: `feat/fundatie` · Complement la `CODE_REVIEW.pdf`

Inventar (~31 teste):

Fișier	Tip	#
<code>HouseCommandServiceTests</code>	unit (Mockito)	8
<code>HouseQueryServiceTests</code>	unit (Mockito)	3
<code>BatteryQueryServiceTests</code>	unit (Mockito)	1
<code>HouseControllerTest</code>	unit	1 (gol)
<code>HouseCommandServiceIT</code>	IT (H2)	7
<code>HouseQueryServiceIT</code>	IT (H2)	6
<code>HouseControllerIT</code>	IT (H2, MockMvc)	5
<code>SolarSyncBrokerApiApplicationTests</code>	context	1

Verde pe hârtie, dar acoperirea e concentrată **exclusiv pe house** și ratează exact zonele cu bug-uri din `CODE_REVIEW.pdf`.

Legendă severitate

- Gol de acoperire / test care ascunde un bug
- Calitatea testelor existente

Găuri care contează

1. Securitatea nu e testată deloc — `addFilters = false`

```
@AutoConfigureMockMvc(addFilters = false) // dezactivează tot lanțul de filtre
```

`@WithMockUser(authorities = {...})` verifică doar `@PreAuthorize` (method security, prin AOP). Lanțul real — `JWTTokenProvider`, resource server, `JWTAuthenticationEntryPoint` (401), `SecurityAccessDeniedHandler` (403) — are **zero teste**. Lipsesc:

- „fără token → 401”
- „user fără `house:manage` → 403”
- „user străin pe casa altuia → 403” **prin HTTP**

Bug-ul #1 din `CODE_REVIEW.pdf` (ownership din `?email`) nu e prins de niciun test, iar `getHouseById` (metoda periculoasă) nu are niciun test de controller. Un singur IT „user străin → 403” pe `/houses/{id}` l-ar fi expus.

2. `HouseControllerTest` unit — test gol, fals pozitiv

```
@Test
void createHouse() throws Exception {
} // corp gol → trece mereu
```

Mock-urile `@InjectMocks` / `@Mock` sunt nefolosite. Un test gol e mai rău decât lipsa lui — umflă numărătoarea și dă impresie de acoperire. Șterge-l sau completează-l.

3. `BatteryQueryServiceTests` cimentează bug-ul

Testează doar calea ADMIN fericită. Nu acoperă:

- non-admin → `HouseAccessDeniedHandler` (respingerea)
- baterie inexistentă (aruncă `HouseNotFoundException`, tip greșit — bug #5)
- owner-ul care nu-și poate vedea bateria

Testul „confirmă” comportamentul greșit ca fiind corect. O ramură negativă ar fi semnalat logica inversată.

4. `GlobalExceptionHandler` — zero teste

Niciun test nu verifică că „not found” dă **404** (acum dă 409), că validarea dă **400/422**, sau că body-ul e **JSON** (acum e `toString()`). Un `MockMvc` test pe un endpoint care aruncă `HouseNotFoundException` ar fi arătat instant 409 + string ne-JSON.

● Calitatea testelor existente

5. Assert-uri tautologice (verifici obiectul pe care tu l-ai mutat)

```
when(houseRepository.save(any(House.class))).thenReturn(existingHouse);  
HouseResponse response = houseCommandService.updateHouse(houseId, request);  
assertEquals(updatedName, existingHouse.getName()); // setterul pe obiectul mutat de serviciu
```

`save` întoarce același `existingHouse` pe care serviciul tocmai l-a mutat prin settere → assert-ul e circular. Nu demonstrează că `save` a primit valorile corecte. **Fix:** `ArgumentCaptor<House>` + `verify(houseRepository).save(captor.capture())` și assert pe ce s-a capturat.

6. Interacțiuni neverificate

```
void testDeleteHouseSuccess() {  
    when(houseRepository.existsById(houseId)).thenReturn(true);  
    assertDoesNotThrow(() -> houseCommandService.deleteHouse(houseId)); // nu verifică delete  
}
```

Aproape niciun `verify(...)`. Ștergerea propriu-zisă nu e testată — lipsește `verify(houseRepository).deleteById(houseId)`.

7. „IT” pe H2, nu integrare reală

```
url: jdbc:h2:mem:testdb;MODE=MySQL  
globally_quoted_identifiers: true
```

Backlog-ul cere Testcontainers **MySQL real**. Cu H2 + identificatori quotați ascunzi diferențe reale (cuvinte rezervate, tipuri, Flyway — oricum dezactivat).

`HouseCommandServiceIT` / `HouseQueryServiceIT` dublează exact unit-urile fără să testeze ceva în plus (tranzacții, constrângeri unique DB, cascade owner→house). Numele `IT` induce în eroare.

8. Module întregi netestate

Zero teste pe: `auth` (register/login), `AuthServiceImpl.permissionsForType` (unde OPERATOR/AGENT cad greșit pe ramura USER), `JWTTokenProvider`, `UserCommandServiceImpl`, `UserController`. Inima de securitate a aplicației e neacoperită.

9. `patchHouse` — un singur câmp acoperit

Testezi doar că `name` se schimbă și `pvPower` (null) rămâne. Nu acoperi `ownerId == null` sau celelalte câmpuri → regula „null nu suprascrie” e testată parțial.

Ce e bine

- Structura unit (Mockito) vs IT e clară; naming `testX` / `xReturnsError` consecvent.
- `HouseCommandServiceTests` acoperă corect cele 3 ramuri de create (success / already-exists / user-not-found) și not-found pe update/delete — modelul de urmat pentru restul slice-urilor.
- `HouseQueryServiceIT.getHouseForCallerReturnsError` testează respingerea la nivel de **service** (owner check). Bun — dar trebuie dublat la nivel **HTTP** (vezi #1).
- JaCoCo configurat.

Priorități

1. **#1** — un IT de controller *cu filtre pornite*: 401 fără token, 403 fără permisiune, 403 user străin pe `/houses/{id}`. Prinde bug-ul principal de securitate.
2. **#4** — 3 teste pe `GlobalExceptionHandler` (404 / 422 / body JSON).
3. **#2 + #3** — șterge testul gol; adaugă ramura negativă la baterie.
4. **#5 + #6** — `ArgumentCaptor` + `verify` în unit-urile de command.
5. Extinde pe `auth` + `permissionsForType`.



MyCodeSchool

Material didactic MyCodeSchool

mycodeschool.ro